How Models Like Claude Opus Are Trained to Excel at SWE-Bench

The Training Pipeline (Four Stages)

Frontier coding models are trained in four distinct stages, each building on the last. Not every model uses all four — but the most capable ones do.

Stage 1	Stage 2	Stage 3	Stage 4
GENERAL PRE-TRAINING	→ CODE-FOCUSED PRE-TRAINING (optional)	→ INSTRUCTION FINE-TUNING (SFT)	→ REINFORCEMENT LEARNING (RLHF / RLVR)
Trillions of tokens: web, books, papers, code, forums	Curated code corpora: GitHub, docs, Stack Overflow, quality-filtered repos	Human-written (instruction, response) pairs for coding tasks	Train against verifiable rewards: code execution, test pass/fail, human preferences
Result: Broad language understanding	Result: Deep code pattern knowledge	Result: Follows coding instructions well	Result: Reasons through problems, self-corrects, produces verified solutions

---

Stage 1: General Pre-Training

Objective: Next-token prediction on a massive, diverse text corpus (trillions of tokens).

What the model learns for code:

- Syntax and semantics of dozens of programming languages
- Common algorithms, data structures, and design patterns
- API conventions, library usage patterns, and framework idioms
- The relationship between natural language descriptions and code implementations
- How documentation, comments, tests, and code relate to each other

Key detail: The model learns these as **statistical patterns**, not explicit rules. It hasn't been told "this is Python syntax" — it's learned that certain token sequences co-occur with extremely high probability. The depth of these patterns, across billions of code examples, is what makes the model appear to "understand" code.

Cost: Tens to hundreds of millions of dollars. Months of training on thousands of GPUs/TPUs.

Stage 2: Code-Focused Continued Pre-Training (optional)

Objective: Continue pre-training on a curated, high-quality code corpus to deepen coding capability beyond what general pre-training provides.

Data curation is the secret weapon here:

- **Hierarchical filtering:** Coarse-to-fine extraction of high-quality code from repositories — removing duplicates, low-quality files, auto-generated code, and test-contaminated data
- **Quality signals:** Files with good documentation, meaningful variable names, proper error handling, and passing CI are weighted higher
- **Decontamination:** Benchmark test cases (HumanEval, SWE-bench, etc.) are explicitly removed to prevent the model from memorizing answers
- **Multi-language coverage:** Code across 50-100+ programming languages, weighted by real-world usage
- **Context-rich data:** Not just isolated functions but full files, multi-file repositories, commit diffs, pull requests with descriptions, and issue → patch pairs

Special training objectives:

- **Fill-in-the-Middle (FIM):** The model learns to predict missing code given surrounding context — critical for code completion and patch generation
- **Repository-level context:** Training on entire repo structures so the model learns file relationships, import graphs, and cross-file dependencies

This stage is what separates a "general model that can code" from a "code-specialized model." It deepens the statistical patterns specifically around code.

Stage 3: Supervised Instruction Fine-Tuning (SFT)

Objective: Transform the base model (a text completer) into an assistant that follows coding instructions.

Training data consists of high-quality (instruction, response) pairs:

- "Fix the bug in this function" → [correct patch]
- "Write unit tests for this class" → [comprehensive test suite]
- "Explain what this code does" → [clear explanation]
- "Refactor this to use the strategy pattern" → [clean refactored code]

Critical for SWE-bench specifically:

- Pairs that demonstrate navigating a codebase, understanding an issue description, identifying relevant files, and producing a minimal correct patch

- Examples of reading test files to understand expected behavior
- Examples of multi-step reasoning: "The test expects X, the code does Y, the fix is Z"

This stage also teaches:

- When to ask for clarification vs. act
- How to structure explanations alongside code
- How to use tools (search, file read, bash execution) effectively
- The format and style of being a helpful coding assistant

Key insight: SFT teaches *format and style* — how to behave as an assistant. But it has a fundamental limitation: it teaches the model to **imitate** correct-looking outputs. The model learns to produce code that looks like the training examples, but SFT alone doesn't teach the model to verify whether its code actually works.

Stage 4: Reinforcement Learning (the critical differentiator)

This is where the biggest capability leaps happen for coding, and it's the stage that most directly targets SWE-bench performance.

The fundamental insight

Pre-training teaches the model what code **looks like** (textual patterns). But code correctness is determined by **execution semantics** — whether it produces the right output when run. There's a gap between "looks right" and "is right." RL bridges this gap.

RLHF (Reinforcement Learning from Human Feedback)

1. The model generates multiple code solutions for a problem
2. Human experts rank which solutions are better (more correct, more readable, more idiomatic)
3. A reward model learns to predict human preferences
4. The coding model is optimized via RL (typically PPO or DPO) to maximize the reward

This teaches the model human-aligned preferences: clean code, good error handling, appropriate comments, idiomatic patterns.

RLVR (Reinforcement Learning with Verifiable Rewards) — the 2025 breakthrough

This is the stage that defined 2025's capability progress. Unlike RLHF, which requires expensive human labels, RLVR uses **automatically verifiable rewards** — primarily test execution.

How it works for code:

1. Model generates a code solution

2. The solution is **executed** in a sandboxed environment against test cases
3. The reward signal is deterministic: tests pass (reward) or fail (no reward)
4. The model is optimized to produce code that passes tests

Why this is so powerful:

- **Scalable:** No human labelers needed — test execution is automated
- **Objective:** Can't be gamed the way subjective ratings can
- **Allows long optimization:** RLHF is a thin stage (limited by human data). RLVR can run for much longer because the reward signal is unlimited
- **Develops reasoning:** The model spontaneously learns to break problems down, check intermediate steps, and backtrack when stuck — because these strategies lead to higher rewards

The key Karpathy observation: Most of 2025's capability progress came from labs "chewing through the overhang" of RLVR — running much longer RL training against verifiable rewards. This is why models got dramatically better at coding without getting proportionally bigger.

What the model learns during RL that it didn't learn from pre-training:

- **Self-verification:** Checking its own work before committing to an answer
- **Backtracking:** "Wait, that approach won't work because..." and trying a different strategy
- **Multi-step planning:** Breaking complex problems into verifiable sub-steps
- **Error recovery:** Recognizing when a solution fails a test case and diagnosing why
- **Minimal patches:** Learning that the smallest correct change is better than a large refactor (directly relevant to SWE-bench)

How This Specifically Targets SWE-Bench Performance

SWE-bench tasks require the model to:

1. Read a GitHub issue description
2. Navigate a real codebase (multiple files, complex dependencies)
3. Identify which files need to change
4. Understand the existing code and tests
5. Produce a minimal, correct patch
6. Have the patch pass the repository's test suite

Each training stage contributes differently:

SWE-bench Requirement	Training Stage	How It Helps
Understanding code in any Python repo	Stage 1 (Pre-training)	Broad exposure to millions of Python repos
Understanding repo structure, imports, dependencies	Stage 2 (Code pre-training)	Repository-level context training
Following issue descriptions as instructions	Stage 3 (SFT)	Instruction-following fine-tuning
Producing correct patches, not just plausible-looking ones	Stage 4 (RLVR)	Execution-based reward signal
Navigating codebase to find relevant files	Stage 3 + 4	Tool-use training (search, read, grep)
Generating minimal, targeted changes	Stage 4 (RL)	RL rewards smaller correct patches over large refactors
Self-correcting when tests fail	Stage 4 (RL)	Learned reasoning and backtracking strategies
Multi-step reasoning about cause and effect	Stage 4 (RL)	Extended thinking / chain-of-thought development

---

The Agentic Scaffolding Layer (not model training, but critical)

SWE-bench scores are achieved by **model + scaffolding** together. The raw model produces better patches, but the harness determines how well those patches integrate into the workflow:

- **Agentic search:** The harness (like Claude Code) provides tools for the model to search the codebase, read files, and understand structure — the model decides *which* tools to use
- **Extended thinking:** Giving the model more inference-time compute (thinking tokens) to reason through complex issues before generating a patch
- **Tool-use training:** The model was specifically trained on trajectories of tool use — knowing when to search vs. read vs. edit
- **Verification loop:** The harness can run tests after patching and feed failures back for the model to retry

This is why the same model can score very differently depending on the scaffolding. A well-designed harness amplifies the model's trained capabilities.

The Benchmark Contamination Caveat

An honest discussion of SWE-bench training must mention contamination concerns. Research has revealed:

- **Solution leakage:** ~60% of successfully resolved SWE-bench issues have solutions directly provided or strongly hinted at in the issue description
- **Weak test validation:** ~48% of resolved issues pass via patches that satisfy weak test cases but may not be truly correct
- **Memorization risk:** Models may have seen SWE-bench tasks (or very similar code) during pre-training, inflating scores beyond genuine problem-solving ability
- **Benchmark "gaming":** As Karpathy noted, "training on the test set is a new art form" — teams construct training data adjacent to benchmark distributions

When SWE-bench+ (a harder, decontaminated version) was applied, resolution rates dropped from ~50% to ~26% on average. This suggests real capability is improving, but headline scores overstate the degree of progress.

Practical takeaway for engineers: The model's coding capability is real and substantial — but SWE-bench scores are not a reliable measure of how well the model will perform on *your* codebase. Your codebase, your patterns, and your edge cases are different from the benchmark distribution. This is why CLAUDE.md, verification, and human review remain essential.

Summary: The Full Training-to-Deployment Stack

```
DATA CURATION
  Trillions of tokens, quality-filtered, decontaminated
      │
      ▼
STAGE 1: GENERAL PRE-TRAINING
  Next-token prediction → Broad knowledge of language + code
      │
      ▼
STAGE 2: CODE PRE-TRAINING (optional)
  Curated code corpus, FIM, repo-level context → Deep code patterns
      │
      ▼
STAGE 3: SUPERVISED FINE-TUNING (SFT)
  Human-written (instruction, response) pairs → Instruction following
      │
      ▼
STAGE 4: REINFORCEMENT LEARNING (RLHF + RLVR)
  Execute code against tests, human preference ranking
  → Self-verification, reasoning, correct solutions
      │
      ▼
SAFETY & ALIGNMENT
```

Constitutional AI, red-teaming, safety classifiers
→ Refusals, guardrails, responsible behavior
|
▼
AGENTIC HARNESS (Claude Code)
Tools, context management, permissions, subagents
→ Autonomous coding agent
|
▼
YOUR CODEBASE
CLAUDE.md, verification, human review
→ Production-quality software

The model that arrives on your machine is the product of all these stages working together. Understanding each stage helps you understand *why* the model behaves the way it does — and how to work with its strengths while compensating for its limitations.